Project 30 – Adaptive Communication Network with Failure Recovery

UDP socket application over an SDN-controlled Mininet network · SDN–Socket Programming Jackfruit Mini Project · Team 4: Podili Sethu Sriram (PES1UG25AM555), Suryakoti Sunil Salunke (PES1UG25AM529), Vishal Sharanabasappa Dindi (PES1UG26AM823) · Repository: https://github.com/sethu-sriram/adaptive-communication-network · 26 October 2026

Abstract

We built a UDP client/server application that runs over a Mininet network with two disjoint paths, steered by an OS-Ken (OpenFlow 1.3) controller. The controller learns about failures in two ways: from OpenFlow link events (hard failures) and from reports sent by the application itself over a UNIX socket (silent failures, such as 100 % packet loss while the interface stays up). The client independently recovers lost datagrams with ACKs, timeouts and retransmissions. We compared this adaptive design with a static baseline (path A only, never reacts) in 90 experiments (2 modes × 3 scenarios × 3 loads × 5 repetitions). In both failure scenarios the adaptive design delivered 100 % of the packets, against about 70.6 % for the baseline, and restored service in 124 ms (link down) and 390 ms (black-hole), against about 10.2–10.3 s for the baseline, which only works again when the failure itself ends. Without failures the two modes are indistinguishable.

1. Problem and objectives

Project 30 asks for an adaptive communication network that keeps a UDP application running when part of the network fails. A network that sends traffic over a fixed path loses all traffic as soon as one link on that path stops working, and the application can do little about it. In a software-defined network the controller can move traffic to another path, but only if it finds out in time.

Objectives

1. A UDP application with application-level recovery (acknowledgements, retransmission) that supports several clients.
2. A primary path (A) and a backup path (B), with a controller that reroutes quickly when the primary path fails.
3. Detection of failures that OpenFlow cannot see, by letting the application tell the controller what it observes (the socket–SDN integration).
4. A quantitative comparison with a static baseline under normal operation, link failure and silent failure, at several loads.

2. System architecture

2.1 Topology

```
          +---- s2 ----+            Path A (preferred): h1 - s1 - s2 - s4 - h2
   h1 - s1              s4 - h2     Path B (backup)   : h1 - s1 - s3 - s4 - h2
          +---- s3 ----+            inter-switch links: 10 Mbit/s, 2 ms delay
```

The topology has two hosts (h1 client, h2 server) and four Open vSwitch switches speaking OpenFlow 1.3. Port numbers follow the link creation order in `topology.py`, and the controller depends on them.

Switch	Port 1	Port 2	Port 3
s1	h1	s2	s3
s2	s1	s4	–
s3	s1	s4	–
s4	s2	s3	h2

2.2 Components

Component	File	Role
Protocol	`protocol.py`	18-byte binary header shared by client and server

Component	File	Role
UDP server (h2)	`server.py`	Receiver thread plus worker pool, ACKs every DATA datagram, per-client statistics
UDP client (h1)	`client.py`	Fixed-rate sender, ACK/timeout/retransmit, outage detection, reports to the controller
Controller	`controller.py`, `run_controller.py`	Path A/B flows, link events, application reports, flow statistics, `static` baseline mode
Topology	`topology.py`	Mininet network with OVS switches and traffic-controlled links
Experiments	`experiment.py`, `run_all.sh`, `analyze.py`, `make_report.py`	Automated failure injection, tables and charts
Tests	`tests/`	Tests of the UDP application and of the controller decision logic that run without Mininet

2.3 Communication flow

Normal operation. The controller installs forwarding rules for path A on s1, s2 and s4. The client sends numbered DATA datagrams to h2, the server answers each with an ACK, and the traffic never touches s3.

Hard failure (interface down).

1. The link s1–s2 goes down and OVS reports the port change.
2. The controller receives `EventLinkDelete`, marks the link down and installs path B.
3. Retransmissions of the lost datagrams reach h2 over path B and are acknowledged.

Silent failure (black-hole, interface still up).

1. The link silently drops every packet, so no OpenFlow event is generated.
2. The client sees consecutive timeouts and no ACK for at least 250 ms, and sends a `PATH_SUSPECT` message to the controller over a UNIX datagram socket.
3. The controller marks the active path suspect and installs path B.
4. The client receives ACKs again and sends `RECOVERED`.
5. After a 20 s hold-down the controller returns to path A.

2.4 Application protocol

Every datagram starts with an 18-byte header in network byte order.

Field	Size	Meaning
magic	2 bytes	`0x5030`, rejects foreign datagrams
type	1 byte	DATA, ACK, FIN or FINACK
attempt	1 byte	0 for the first transmission, increases with each retransmission
client_id	2 bytes	Identifies the client, so one server serves several clients
seq	4 bytes	Sequence number
send_ts_ns	8 bytes	Send timestamp, echoed in the ACK

Because the ACK echoes the attempt number and timestamp, the client measures a round-trip time per attempt without the usual retransmission ambiguity.

3. Implementation

3.1 UDP server

The server owns one UDP socket. A receiver thread puts datagrams on a queue and four worker threads validate them, update per-client statistics (unique packets, duplicates, highest sequence number) and send the ACK. Malformed datagrams are ignored, duplicates caused by retransmission are counted, and a FIN message makes the server print the statistics of that client.

3.2 UDP client and application-level recovery

The client sends `rate × duration` datagrams at a fixed rate and keeps every unacknowledged datagram in a pending table.

- **Retransmission:** timeout 100 ms, multiplied by 1.5 after every attempt, at most 5 retries; after that the packet is declared lost. A packet can therefore be rescued for about 1.4 s.
- **Outage detection:** the client counts consecutive timeouts and measures the time since the last ACK.
- **Controller notification:** after at least 3 consecutive timeouts and at least 250 ms without any ACK, the client sends a JSON `PATH_SUSPECT` message (at most one per second while the outage lasts). The silence condition filters out short stalls of the client process, which on a virtual machine would otherwise cause needless reroutes. When ACKs return, the client sends `RECOVERED`.
- **Output:** a per-packet CSV and a summary JSON per run.

3.3 SDN controller

Flow rules. Rules match `eth_dst` (the host MAC addresses are fixed) at priority 100 and carry the cookie `0x30`. A table-miss rule at priority 0 sends unknown packets to the controller, where they are only logged. Hosts use static ARP entries, so the controller needs no ARP handling.

Path	Towards h2 (out port)	Towards h1 (out port)
A	s1 → 2, s2 → 2, s4 → 3	s4 → 1, s2 → 1, s1 → 1
B	s1 → 3, s3 → 2, s4 → 3	s4 → 2, s3 → 1, s1 → 1

Make-before-break reroute. The controller writes the new path's rules on the middle switch first and on the edge switches last. A rule with an identical match and priority replaces the old one in place, so there is no gap. Afterwards it deletes only its own rules (by cookie) from switches that the new path no longer uses, and never touches the table-miss rule.

Path selection. Path A is preferred. A path is usable if none of its links is known to be down and it is not marked suspect. If every topologically usable path is suspect and a fresh failure report arrives for the active path, the controller rotates to the other path instead of staying on the preferred one.

Failure detection

Source	Failure type	Reaction
`EventLinkDelete` / `EventLinkAdd`	Hard failure	Mark link down or up, reconcile paths
Application report `PATH_SUSPECT`	Silent failure	Mark active path suspect for 20 s, reconcile with rotation
Hold-down expiry	Silent failure over	Remove suspicion, return to path A

Stale-report filter. Every report carries `since`, the send time of the newest timed-out packet. The controller ignores a report if `since` is not later than its last path change (plus 50 ms), so a report about a failure of the old path cannot condemn the new one. Malformed reports and invalid `since` values are logged and ignored.

Monitoring and logging. The controller polls flow statistics every 2 s and logs the packet counters per switch, which shows which path carries the traffic. All events (`LINK_DOWN`, `APP_REPORT`, `PATH_SUSPECTED`, `PATH_INSTALLED`, …) are also written as JSON lines for the experiments.

Static baseline. With `P30_MODE=static` the controller installs path A once and ignores every failure. Reports from the client are logged but not acted upon.

3.4 Socket–SDN integration

Mininet hosts live in separate network namespaces, so the client cannot reach the controller at 127.0.0.1, but both share the filesystem. The controller therefore listens on a UNIX datagram socket (`/tmp/p30_controller.sock`), and the client sends its reports there. The integration works in both directions: the application's observations change the network's forwarding state, and the controller's new rules decide whether the application's ACKs return.

3.5 Experiment automation

`run_all.sh` starts a fresh controller for every run, builds the topology, waits 6 s for link discovery, checks connectivity, starts the server and client, injects the failure at t = 10 s (`link s1 s2 down`, or 100 % loss on both interfaces of the link) and removes it at t = 20 s. `analyze.py` aggregates all runs; `make_report.py` produces the results section from the data.

The figures below show the script at work during a pilot sweep (QUICK=1 REPS=1, 50 packets/s) that was run before the full 90-run sweep. For every configuration the topology is built, the link-discovery wait and the 0 % packet-loss connectivity check are passed, and the failure is injected at t = 10 s and restored afterwards.

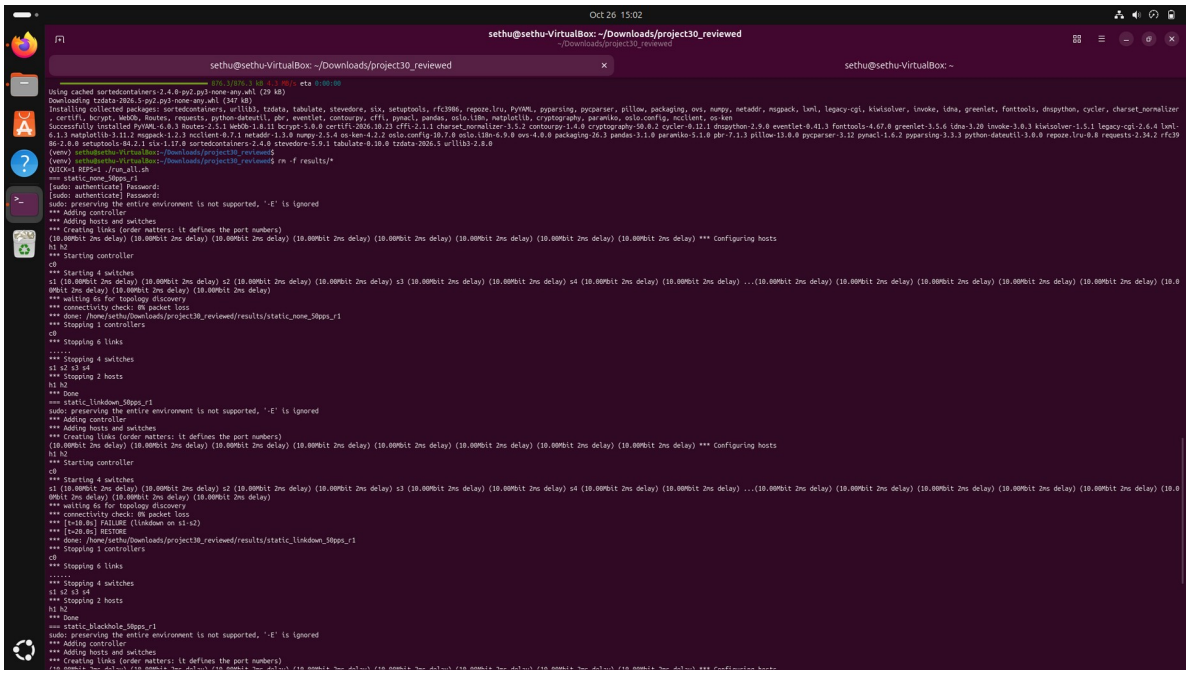


Figure 1. run_all.sh output for the static baseline: static_none_50pps_r1 completes, then static_linkdown_50pps_r1 injects the link-down failure on s1–s2 at t = 10.0 s and restores it at t = 20.0 s.

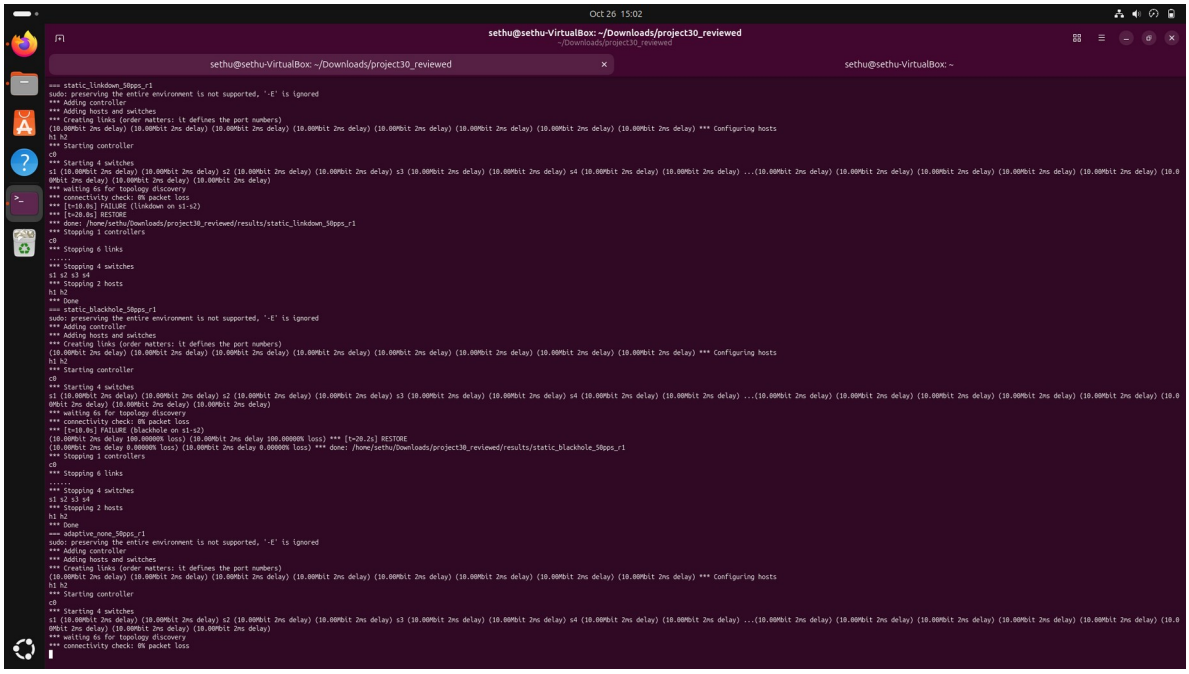


Figure 2. run_all.sh output for static_blackhole_50pps_r1: the black-hole is created with 100 % netem loss on both interfaces of s1–s2 (the link stays up) at t = 10.0 s and removed at t = 20.2 s; adaptive_none_50pps_r1 then starts.

4. Evaluation methodology

Item	Value
Platform	Ubuntu in a VirtualBox virtual machine, Mininet, Open vSwitch, OS-Ken 4.2.2
Run length	30 s, failure from t = 10 s to t = 20 s on link s1–s2 (path A)
Loads	20, 50 and 100 packets/s, 64-byte payload
Scenarios	`none`, `linkdown`, `blackhole`
Approaches	static baseline and adaptive
Repetitions	5 per configuration, 90 runs in total
Statistics	Mean ± standard deviation; a pooled adaptive–static difference is called significant if its Welch-style z-value exceeds 2

Data audit. Before the final analysis the saved data were checked. runs.csv holds exactly 90 runs, five for each combination of scenario, load and mode, and the recovery time is missing only for the no-failure runs, where there is nothing to recover from. A per-packet CSV (adaptive, black-hole, 50 packets/s, repetition 1) has 1 500 rows (30 s × 50 packets/s), no missing values, and attempt counts consistent with the retransmission logic: 1 475 packets were sent once, 15 twice, 7 three times and 3 four times.

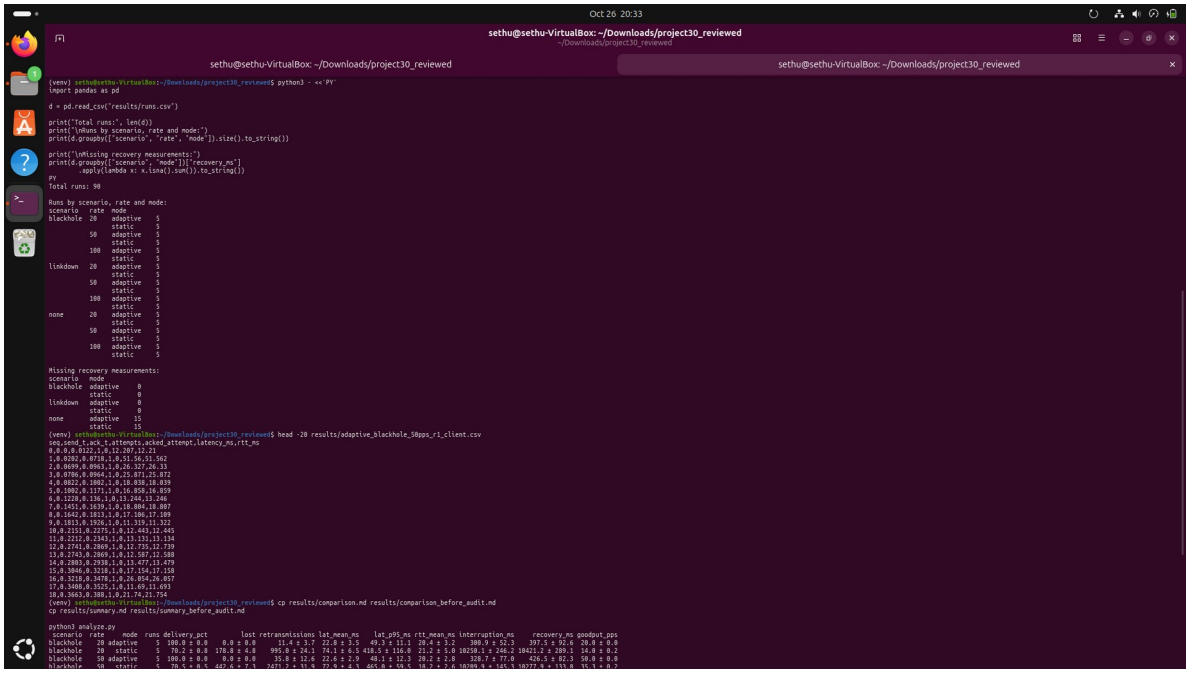


Figure 3. Data audit: 90 runs in total, 5 per scenario/load/mode; recovery time is missing only for the no-failure (none) runs; the first rows of one per-packet CSV.

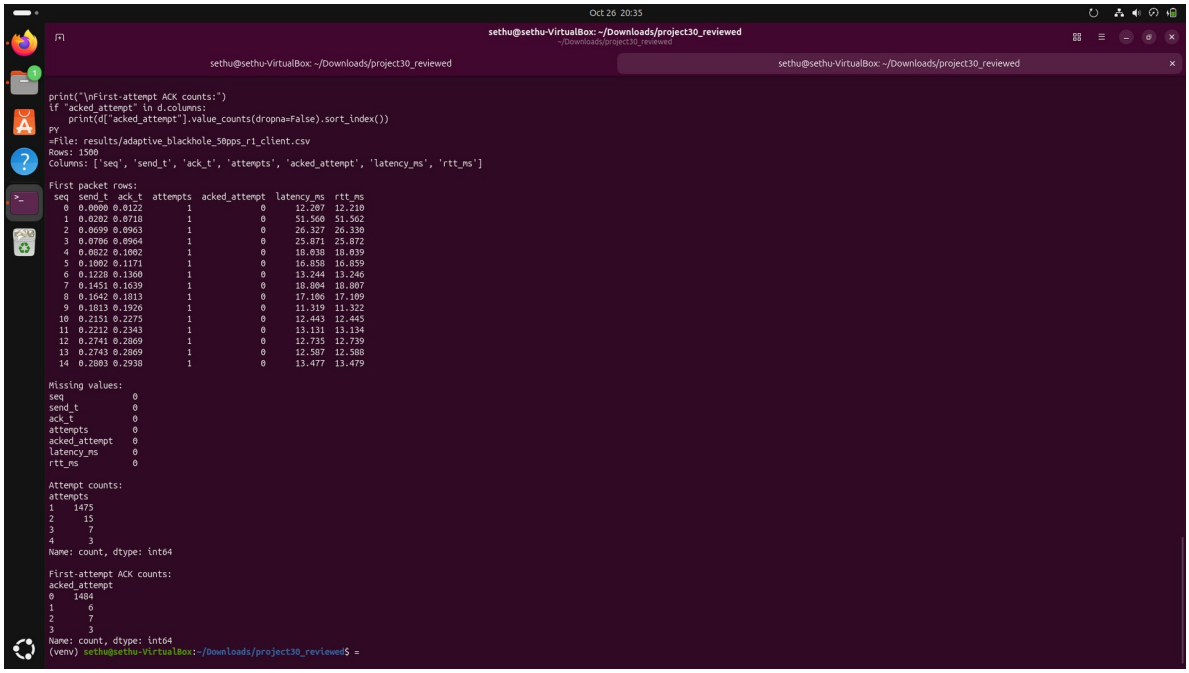


Figure 4. Data audit of one per-packet CSV: 1 500 rows, no missing values, and the distribution of attempts and of the attempt on which each packet was acknowledged.

Metrics

- **Delivery ratio:** acknowledged packets divided by sent packets.
- **Retransmissions:** total retransmitted datagrams per run (the cost of recovery).
- **p95 latency:** 95th percentile of the time from first send to ACK.
- **Recovery time:** time from the failure until a *sustained* recovery. The first packet affected by the failure is found; the recovery packet is the first later packet that is acknowledged on its first attempt and is followed, within 0.5 s, by first-attempt successes for at least 80 % of the packets. The time is measured from the failure timestamp to that packet's ACK.
- **Longest ACK gap:** longest interval without any ACK (a reference value only).

5. Results

5.1 Adaptive versus static (all loads pooled; 15 runs per scenario/mode)

Scenario	Metric	Adaptive	Static	Adaptive − static	n (e
none	delivery_pct	100.00	100.00	0.00	15
none	lat_p95_ms	50.24	48.32	1.91	15
none	retransmissions	22.20	11.40	10.80	15
linkdown	delivery_pct	100.00	70.80	29.20	15
linkdown	recovery_ms	123.56	10173.77	-10050.21	15
linkdown	lat_p95_ms	42.49	502.09	-459.60	15
linkdown	retransmissions	16.93	2802.67	-2785.74	15
blackhole	delivery_pct	100.00	70.40	29.60	15
blackhole	recovery_ms	389.76	10319.79	-9930.03	15
blackhole	lat_p95_ms	53.80	457.12	-403.32	15
blackhole	retransmissions	47.00	2808.00	-2761.00	15

In both failure scenarios the adaptive–static differences in delivery, recovery time, p95 latency and retransmissions are significant; in the no-failure scenario none of the differences is.

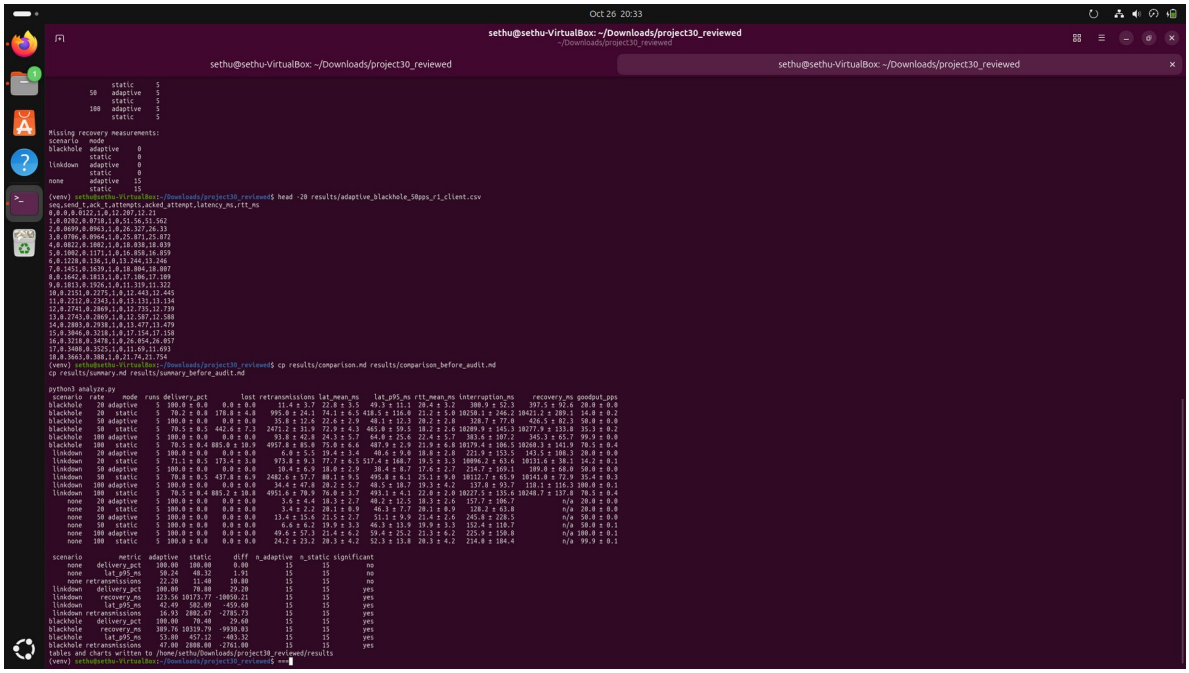


Figure 5. Final analysis output after the data audit: summary table and adaptive-versus-static comparison. The comparison rows are the values in Table 5.1 (for example black-hole recovery 389.76 ms adaptive and 10 319.79 ms static).

5.2 Results per load – link down (mean ± standard deviation; five repetitions per configuration)

The per-load tables below retain the values shown in the final detailed experiment output; each row is an average across five repetitions.

pps	Mode	Delivery (%)	Lost	Retransm.	p95 (ms)	Recovery (ms)
20	adaptive	100.0 ± 0.0	0	6 ± 6	41 ± 9	144 ± 108
20	static	71.1 ± 0.5	173 ± 3	974 ± 9	517 ± 169	10 132 ± 38
50	adaptive	100.0 ± 0.0	0	10 ± 7	38 ± 9	109 ± 68
50	static	70.8 ± 0.5	438 ± 7	2 483 ± 58	496 ± 6	10 141 ± 73
100	adaptive	100.0 ± 0.0	0	34 ± 48	48 ± 19	118 ± 116

pps	Mode	Delivery (%)	Lost	Retransm.	p95 (ms)	Recovery (ms)
100	static	70.5 ± 0.4	885 ± 11	4 952 ± 71	493 ± 4	10 249 ± 138

5.3 Results per load – black-hole (mean ± standard deviation; five repetitions per configuration)

pps	Mode	Delivery (%)	Lost	Retransm.	p95 (ms)	Recovery (ms)
20	adaptive	100.0 ± 0.0	0	11 ± 4	49 ± 11	398 ± 93
20	static	70.2 ± 0.8	179 ± 5	995 ± 24	419 ± 116	10 421 ± 289
50	adaptive	100.0 ± 0.0	0	36 ± 13	48 ± 12	426 ± 82
50	static	70.5 ± 0.5	443 ± 7	2 471 ± 32	465 ± 60	10 278 ± 134
100	adaptive	100.0 ± 0.0	0	94 ± 43	64 ± 26	345 ± 66
100	static	70.5 ± 0.4	885 ± 11	4 958 ± 85	488 ± 3	10 260 ± 142

The complete tables, including the no-failure runs, are in `results/summary.md` and `results/comparison.md`.

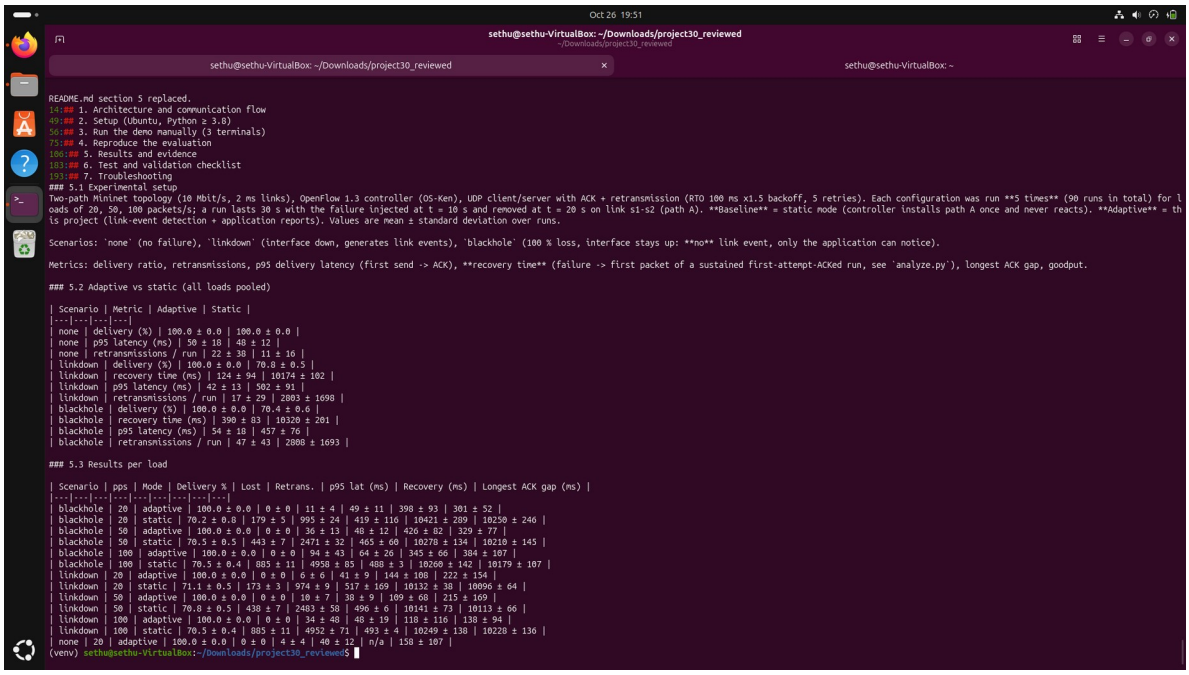


Figure 6. Results section regenerated into README.md by make_report.py --update-readme from the same runs.csv; the per-load values agree with the tables in sections 5.2 and 5.3.

5.4 Charts

Figures below are embedded from the supplied results/ directory. They visualize the same analysed run set summarized in results/summary.md and results/comparison.md. Recovery time is based on sustained first-attempt ACK success; longest ACK gap is reported separately and is not treated as recovery time.

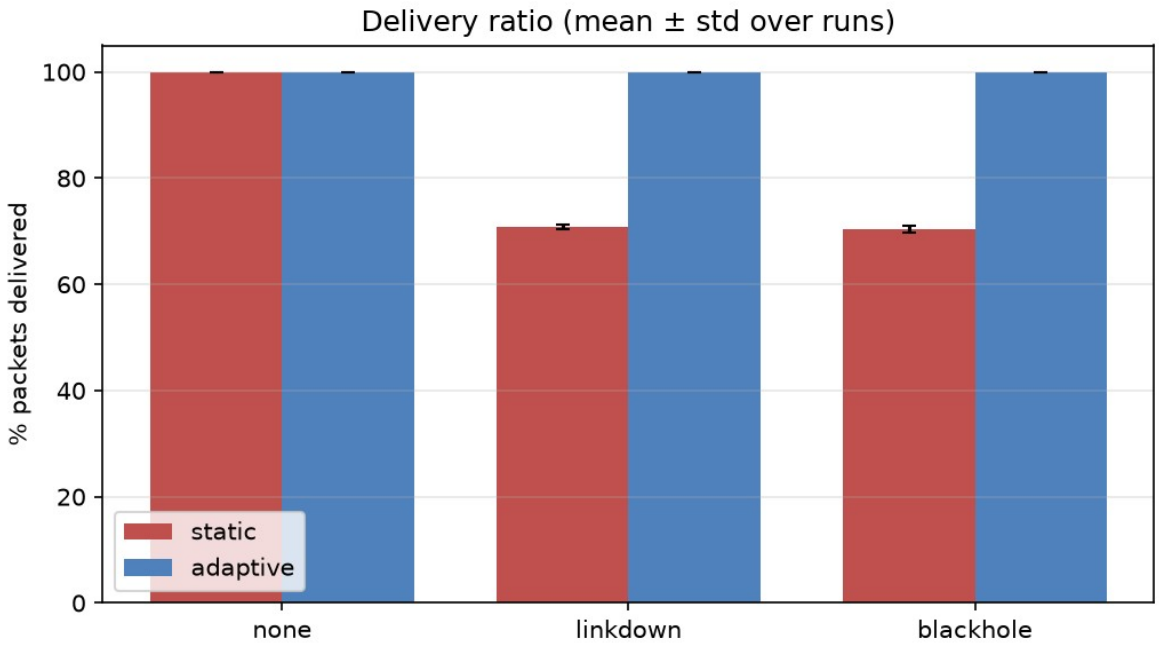


Figure 7. Packet delivery ratio by scenario and mode (mean ± standard deviation).

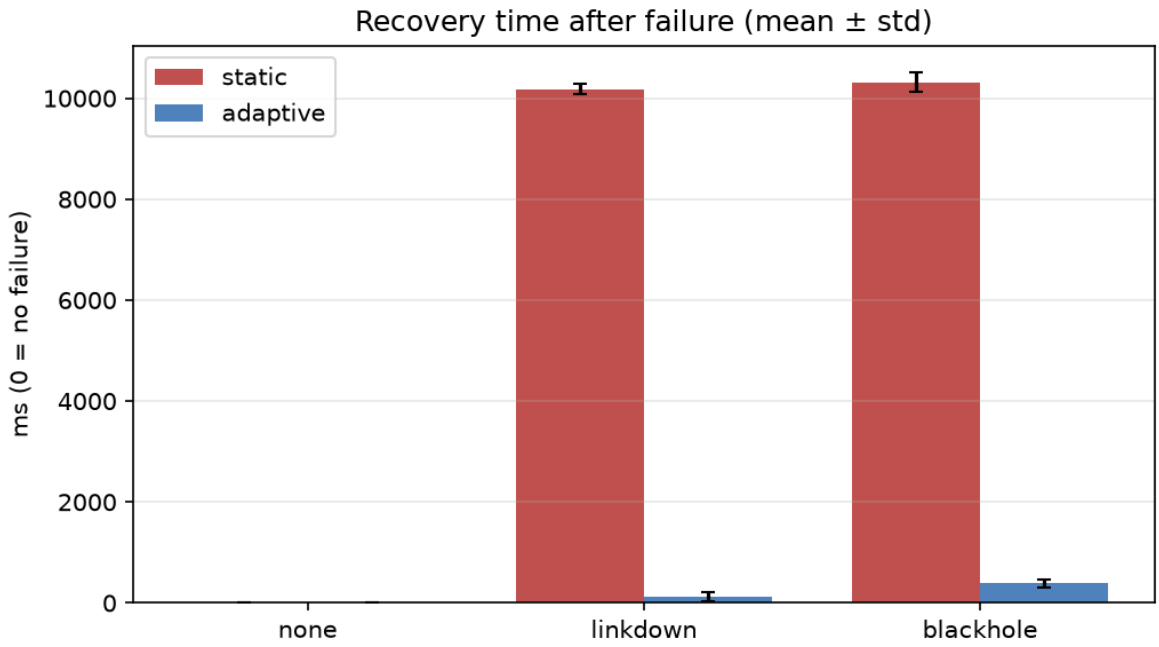


Figure 8. Recovery time after injected failures (mean ± standard deviation).

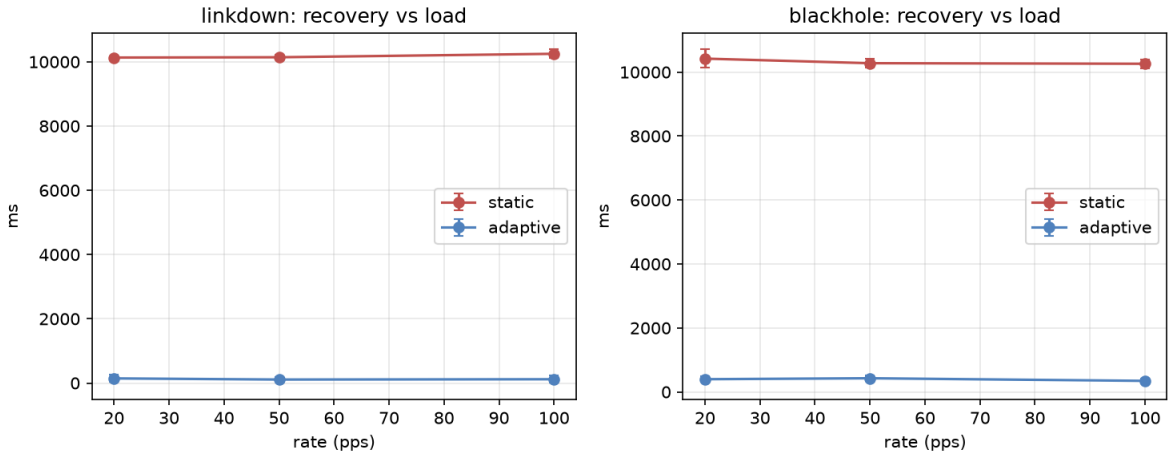


Figure 9. Recovery time across offered loads for link-down and black-hole failures.

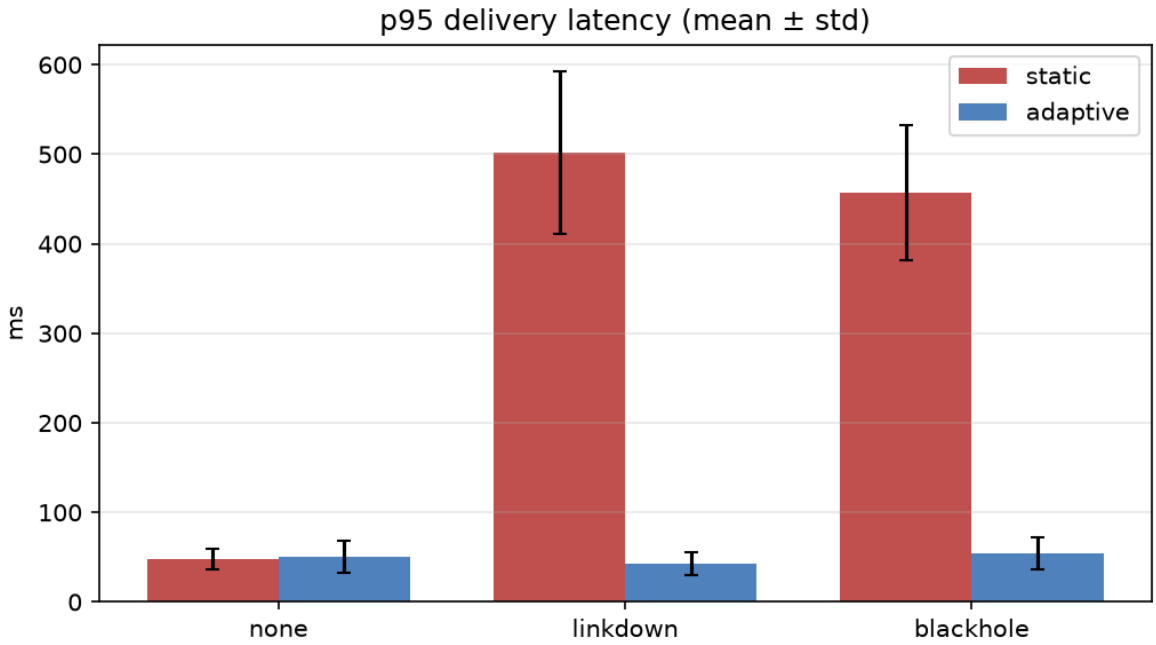


Figure 10. 95th-percentile end-to-end packet latency by scenario and mode.

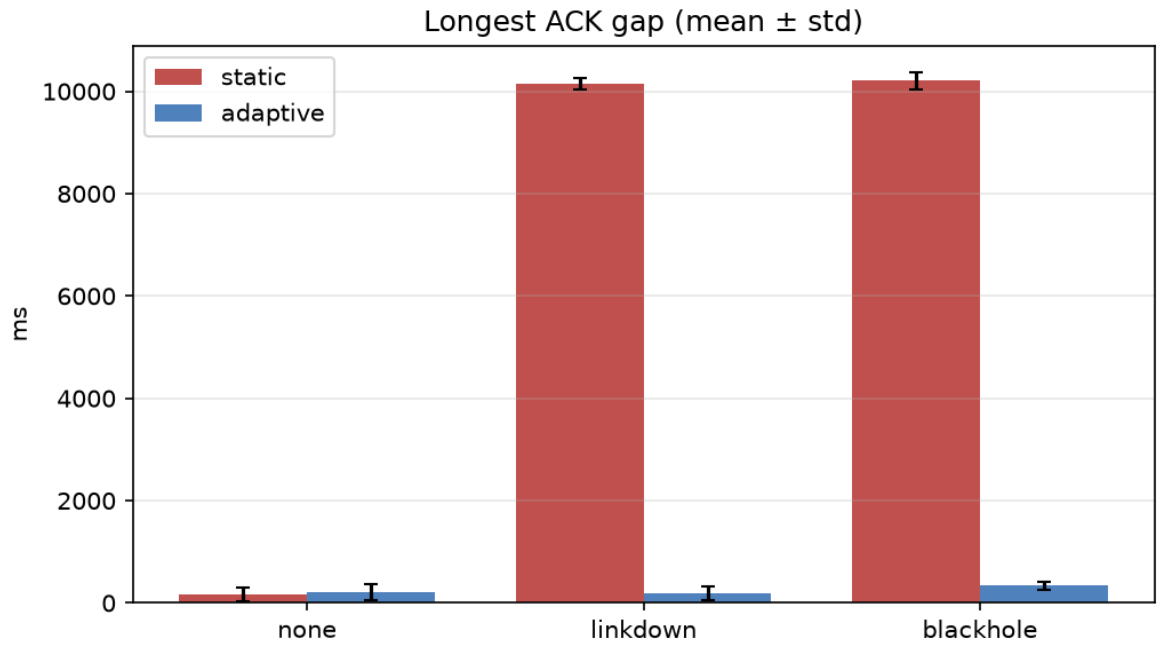


Figure 11. Longest acknowledgement gap; shown separately from the sustained-recovery metric.

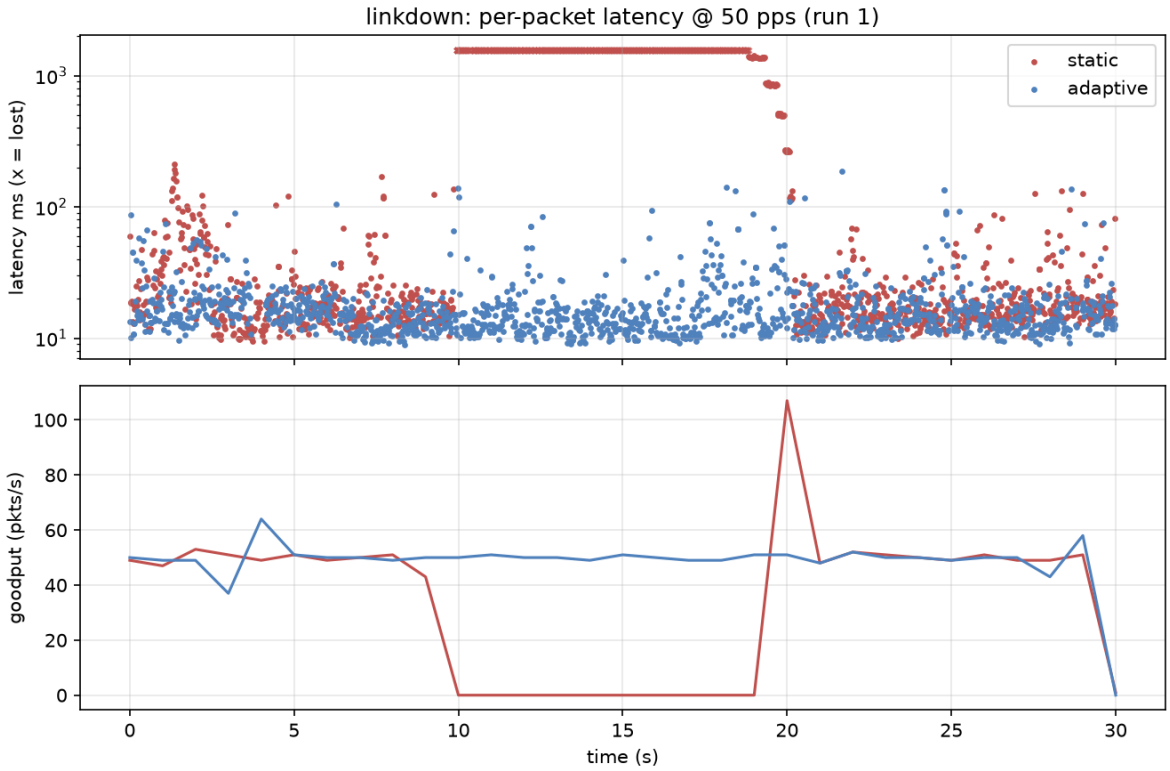


Figure 12. Packet-level timeline for link-down failure and recovery.

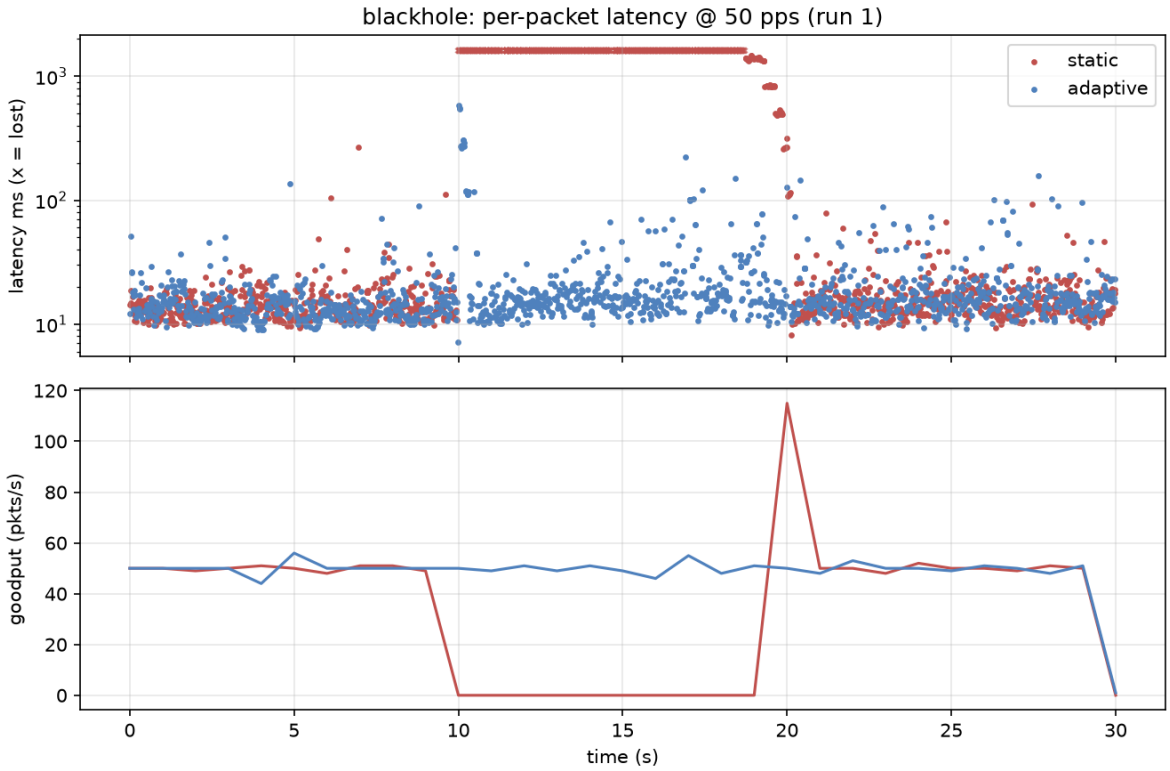


Figure 13. Packet-level timeline for silent black-hole failure and recovery.

6. Discussion

1. **Adaptive recovery works for both failure types.** With the link down, recovery takes about 124 ms, roughly 80 times faster than the baseline; with the black-hole it takes about 390 ms, roughly 26 times faster. The baseline only recovers when the failure ends, so its recovery time equals the failure duration (10 s) plus a small margin.
2. **Why the baseline delivers about 70 %.** Packets sent during the 10 s failure are lost once their retries run out after about 1.4 s. Only packets sent during the last 1.4 s of the failure are rescued by retransmission after the link returns. That leaves roughly (30 − 10 + 1.4) / 30 ≈ 71 % of the packets, which matches the measured 70.4–71.1 %.
3. **Detection path matters.** Link events recover about 270 ms sooner than application reports, because the client first needs three consecutive timeouts and 250 ms of silence before it reports. In the controller logs the new path was installed on average 73 ms after a link failure (range 18–237 ms) but 346 ms after a black-hole began (range 261–451 ms). This is the price of detecting failures that OpenFlow cannot see at all.
4. **No cost in normal operation.** Without failures both modes deliver 100 % of the packets, and the differences in latency and retransmissions are not significant. They stay within the jitter of the virtual machine. One caveat: in 3 of the 15 adaptive no-failure runs a client report, sent although nothing had failed, moved the traffic to path B; delivery stayed at 100 % because path B works, so this cost nothing here, but it shows that reports can be spurious.
5. **Cost of recovery grows with load but stays small.** In the black-hole scenario the adaptive design needs on average 11, 36 and 94 retransmissions at 20, 50 and 100 packets/s, while the baseline needs 995, 2 471 and 4 958 and still loses about 30 % of the packets.
6. **Make-before-break pays off.** The controller installs a reroute in a few milliseconds (median 2.7 ms over the 46 reroutes logged during the failure windows; nine outliers took 8–66 ms), so the recovery time is dominated by failure detection and by the client's retransmission timeout, not by flow installation.

7. Issues found during development and evaluation

These problems were found by running the system, and each one changed the design or the analysis.

#	Observation	Cause	Fix
1	The initial controller handled only	Delete of all flows, link check	Per-path link tracking, cookie-

#	Observation	Cause	Fix
	some link failures and its cleanup also removed the table-miss rule	limited to two links	based deletion of the controller's own rules
2	`osken-manager` was not installed with OS-Ken 4.2.2	The version ships no command-line entry point	`run_controller.py` launches the same application manager
3	First black-hole run: delivery 89.8 %, 4.35 s gap, controller stuck on the failed path	A short stall suspected path B as well; with both paths suspect the controller fell back to the preferred path A, which was down	When all usable paths are suspect, rotate to the other path on every fresh report
4	Spurious reports from short client stalls	Reports were triggered by timeout count alone	Additional condition: no ACK for 250 ms
5	Three of 15 static black-hole runs showed a recovery of 54–129 ms although they lost 30 % of the packets	A few packets slip through while `tc` is reconfigured, and the first metric accepted a single first-attempt success	Recovery now requires a sustained first-attempt success (80 % within 0.5 s); all runs re-analysed from the saved packet logs
6	One adaptive link-down run (50 packets/s, repetition 3) showed an 11 s ACK gap and an 11 s latency	The client process or VM stalled; a latency of 11 s is impossible with a 1.4 s retry window	The run was repeated; the original files are kept in `results/discarded/`

Three of the issues left visible evidence in terminal output; the figures below show it.

Issue 3 – controller stuck on the failed path. In the first black-hole run the controller log shows the client's PATH_SUSPECT reports repeating while the controller keeps marking path A suspect and never leaves it. The run ended with 89.8 % delivery and a 4 350 ms interruption instead of full recovery (pilot run, 50 packets/s).

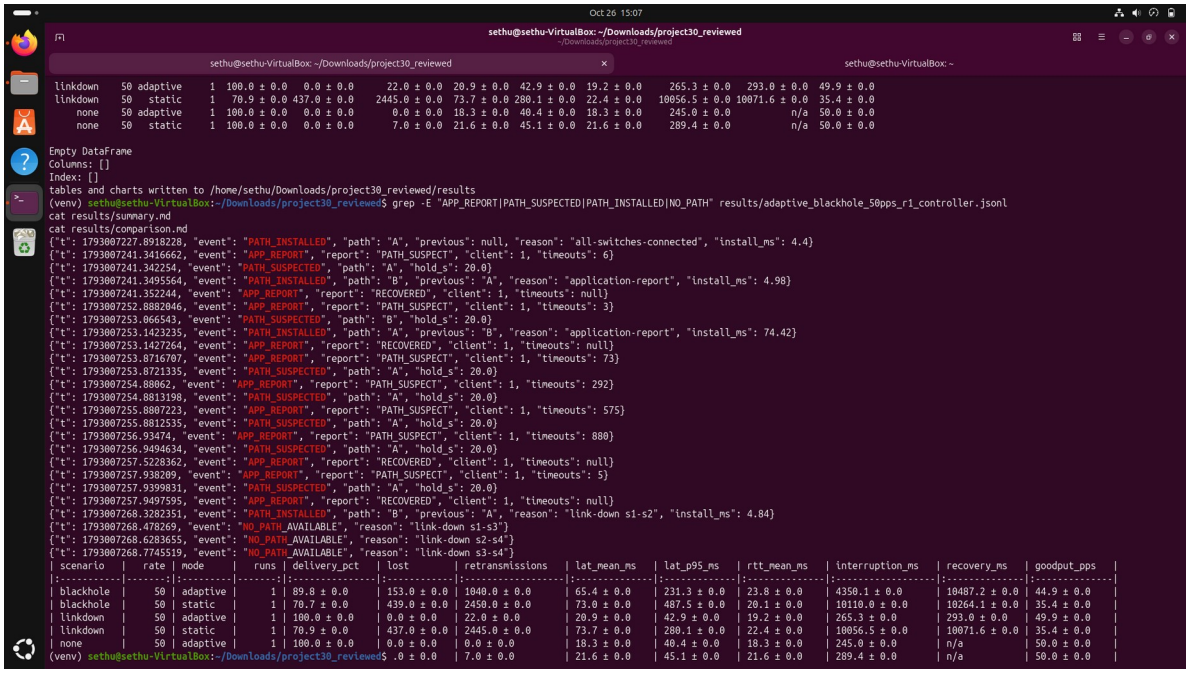


Figure 14. Pilot black-hole run before the fix: repeated APP_REPORT / PATH_SUSPECTED events for path A, and the pilot table row with 89.8 % delivery and a 4 350.1 ms interruption.

Issues 5 and 6 – recovery metric and one outlier run. Listing the adaptive runs with an interruption above 1 s finds exactly one: link-down, 50 packets/s, repetition 3, with an interruption of 11 148 ms and a maximum latency of 11 161 ms. Listing the static black-hole runs shows three with a recovery time of only 54–129 ms although they lost about 30 % of their packets.

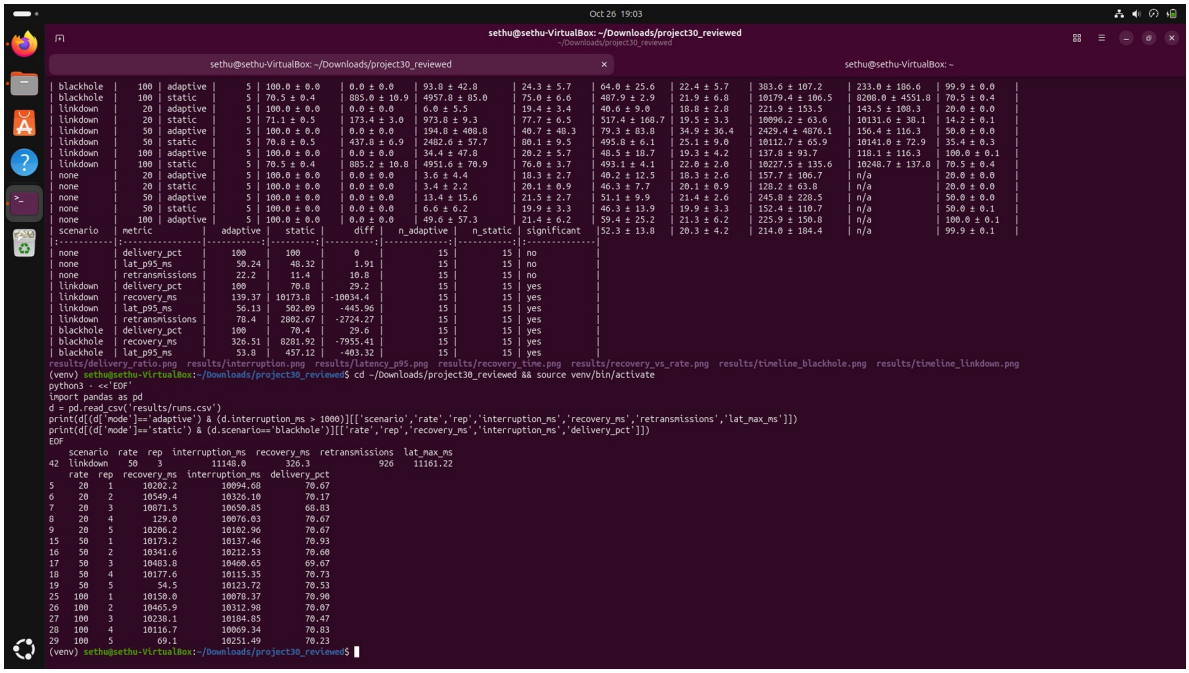


Figure 15. Analysis of runs.csv: the one adaptive run with an interruption above 1 s, and the static black-hole runs (recovery of 129.0, 54.5 and 69.1 ms in three runs despite about 70 % delivery).

Effect on the headline numbers. After the sustained-recovery rule (80 % first-attempt successes within 0.5 s) was added to analyze.py and all runs were re-analysed from the saved packet logs, the pooled black-hole recovery changed from 326.51 ms (adaptive) and 8 281.92 ms (static) to 389.76 ms and 10 319.79 ms. The link-down recovery and all delivery, latency and retransmission figures did not change. The earlier values appear only in the superseded output in Appendix B and must not be quoted.



Figure 16. Re-analysis with the sustained-recovery rule: the static black-hole rows now show recovery times of about 10.3 s, and the pooled black-hole recovery is 389.76 ms (adaptive) versus 10 319.8 ms (static).

8. Testing

Automated tests (no Mininet required)

- `tests/test_loopback.py` runs the real client and server through a relay that drops all packets during a window: no failure (100 % delivery, no retransmission, no report); a 0.5 s outage (100 % delivery through retransmissions, `PATH_SUSPECT` and `RECOVERED` reported); a 3 s outage with 2 retries (packets genuinely lost).
- `tests/test_controller_logic.py` drives the controller with stub switches and checks ten behaviours: boot installs path A with the correct ports; link failure reroutes once and cleans only its own rules; link restoration returns to path A; failure of the unused backup path does not disturb traffic; a double failure does not crash; an application report reroutes and stale reports are ignored; invalid `since` values are ignored; the hold-down expires; with both paths suspect the controller rotates; the static baseline never reacts.

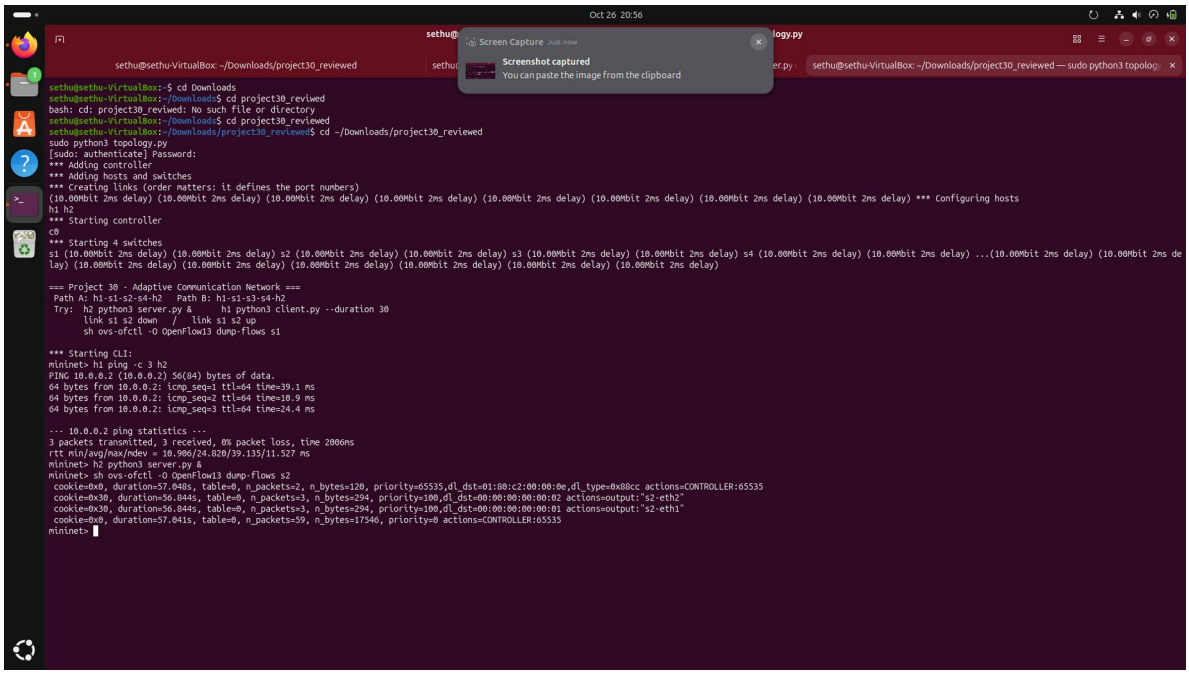


Figure 17. Mininet session: topology start-up, h1 ping h2 with 0 % loss over path A (T1), the server started on h2, and ovs-ofctl dump-flows s2 showing the cookie-0x30 priority-100 rules for both directions plus the table-miss rule (path A is installed on s2).

System test cases

#	Test	Expected	Status
T1	`h1 ping h2` with the controller running	0 % loss over path A	Verified in Mininet
T2	UDP stream without failure	100 % delivery, flow counters grow on s2 only	Verified (`none` runs)
T3	`link s1 s2 down`	`LINK_DOWN`, then `PATH_INSTALLED B`	Verified (controller logs of the `linkdown` runs)
T4	`link s1 s2 up`	`LINK_UP`, then `PATH_INSTALLED A`	Verified
T5	100 % loss on s1–s2	`APP_REPORT`, `PATH_SUSPECTED`, `PATH_INSTALLED B`	Verified (`blackhole` runs)
T6	Failure of s3–s4 while on path A	No reroute, no disturbance	Verified in the controller logic tests
T7	Three concurrent clients	All served, per-client statistics at the server	Verified in Mininet (demo_t7.py, results below)
T8	Both paths down	`NO_PATH_AVAILABLE`, no crash, recovery when a link returns	Verified in the controller logic tests; the Mininet log also showed `NO_PATH_AVAILABLE` when the network was torn down

Controller event traces. The tables and logs below are taken from the saved controller logs of two runs (adaptive, 50 packets/s, repetition 1). Times are measured from the failure injection recorded in the run's metadata file. They provide the evidence for T3, T4, T5 and the Mininet part of T8.

Scenario	Time	Event
Link down	+0.029 s	`LINK_DOWN` s1–s2, active path A
	+0.042 s	`PATH_INSTALLED` B (reason link-down, installed in 0.50 ms)
	+10.568 s	`LINK_UP` s1–s2, active path B
	+10.572 s	`PATH_INSTALLED` A (reason link-up, 3.16 ms)
Black-hole	+0.354 s	`APP_REPORT` PATH_SUSPECT from client 1 after 11 timeouts
	+0.356 s	`PATH_SUSPECTED` path A, hold-down 20 s
	+0.362 s	`PATH_INSTALLED` B (reason application-report, 5.38 ms)
	+0.394 s	`APP_REPORT` RECOVERED from client 1 (ACKs flow again)
	+20.596 s	`SUSPECT_EXPIRED` path A
	+20.599 s	`PATH_INSTALLED` A (reason suspect-expired, 2.11 ms)
Teardown (T8)	+20.693 s to +20.839 s	`LINK_DOWN` on all four inter-switch links while Mininet stops; the last three give `NO_PATH_AVAILABLE` without a crash

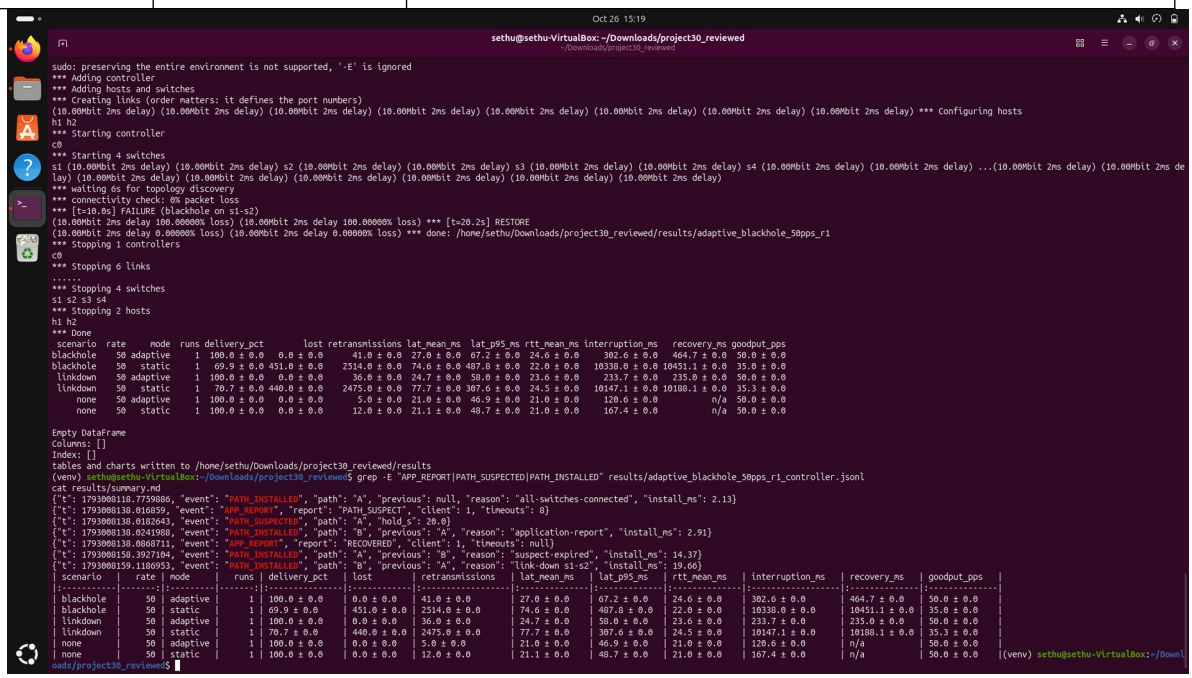


Figure 18. Controller log (JSON lines) of a pilot black-hole run showing the same sequence: APP_REPORT after 8 timeouts, PATH_SUSPECTED, PATH_INSTALLED B, RECOVERED, and later the suspect-expired return to path A. The pilot results table below it is not used elsewhere in this report.

T7 – concurrent clients. The script demo_t7.py starts three clients (IDs 1–3, 20 packets/s, 30 s each) at the same moment on h1, so all of them send through the SDN network to the same server at the same time. In both runs every client delivered all 600 packets and lost none. The server saw the first packets of the three clients within the same second and the last packets within the same second, so the clients really overlapped. In the no-failure run the controller logged no events after the clients started, so no reroute was triggered.

Run	Client	Sent	Delivered	Lost	Server duplicates
No failure	1	600	600	0	0
No failure	2	600	600	0	0
No failure	3	600	600	0	0

Run	Client	Sent	Delivered	Lost	Server duplicates
Link s1–s2 down from t = 10 s to t = 15 s	1	600	600	0	1
Link s1–s2 down from t = 10 s to t = 15 s	2	600	600	0	1
Link s1–s2 down from t = 10 s to t = 15 s	3	600	600	0	0

In the failure run the controller log shows the failure handled as intended (times in seconds after the clients started):

Time (s)	Controller event
10.0	LINK_DOWN s1–s2
10.1	PATH_INSTALLED B (reason: link-down s1–s2)
15.6	LINK_UP s1–s2
15.6	PATH_INSTALLED A (reason: link-up s1–s2)

All three clients delivered every packet. The server received one duplicate datagram from clients 1 and 2 and none from client 3, which fits a packet that was in flight when the link went down being retransmitted; an earlier run of the same test gave one duplicate for each client, so the exact count depends on timing. The three clients kept their own sequence numbers and statistics throughout. An earlier attempt, in which the server had been started only 0.5 s before the clients, showed a client report at the very start of the run that moved traffic to path B before the link failed; it was repeated with a 2 s wait for the server, and the result above is from that repeat.

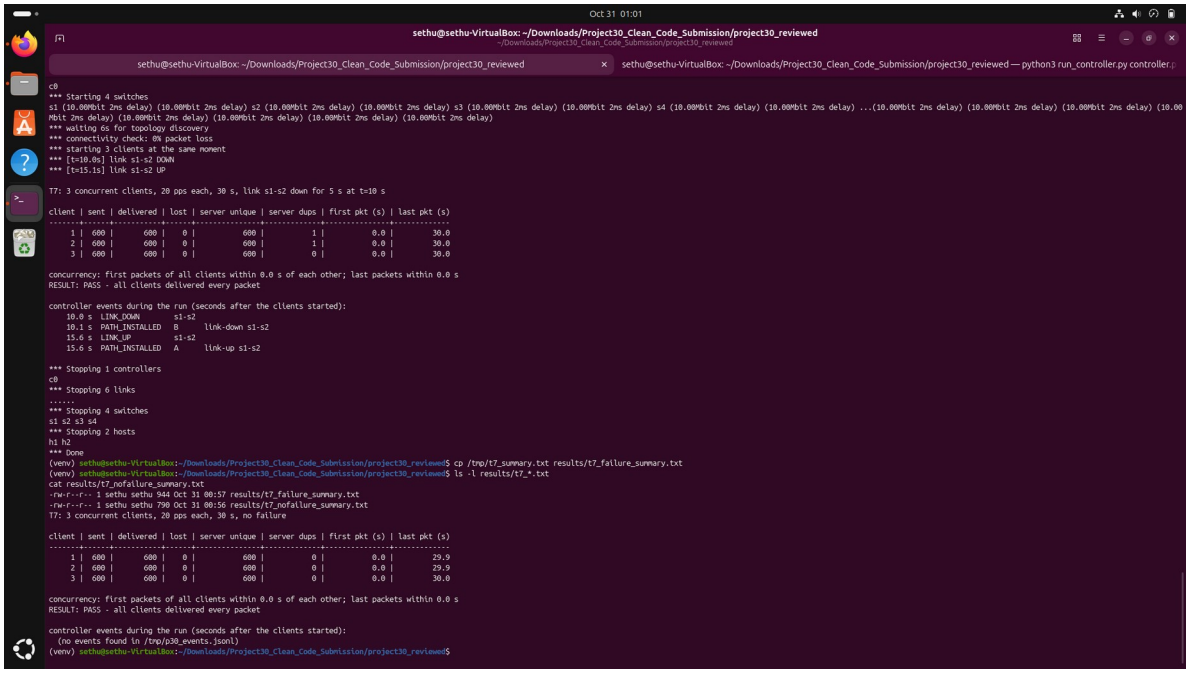


Figure 19. Output of demo_t7.py --fail (three concurrent clients, link s1–s2 down from t = 10 s to t = 15 s, with the controller's event timeline) and, at the bottom, the saved results/t7_nofailure_summary.txt.

9. Limitations and future work

- The application masks an outage only up to about RTO × retries (1.4 s); longer outages depend on the controller's reroute.
- After a silent failure there is no link-up event, so the controller returns to path A only when the 20 s hold-down expires. A shorter hold-down returns sooner but risks flapping between paths.
- Only a single failure on one link was measured. Double failures are covered by the logic tests but not by timed experiments.
- Concurrent operation was verified with three clients (T7), but the timed 90-run experiments use a single client.
- The failure injection (a failure at a fixed time on a fixed link) is deterministic; random or repeated failures, congestion and packet-loss rates below 100 % are future experiments.

- All results come from one virtual machine. Its scheduling jitter shows up as a few retransmissions in the no-failure runs and as large standard deviations in some cells, so absolute values will differ on other machines. The relative comparison is the result.
- The controller trusts reports from any local process. A real deployment would authenticate the application, rate-limit reports and probe the path actively before acting on a report.
- Possible extensions: active probing by the controller, load balancing over both paths, more than two paths, and reporting from the server side as well.

10. Reproducing the results

```
sudo apt install mininet openvswitch-switch
python3 -m venv venv && source venv/bin/activate
pip install os-ken matplotlib pandas tabulate
chmod +x run_all.sh
REPS=5 ./run_all.sh                # 90 runs, about 70 minutes
python3 make_report.py --update-readme
```

Repository layout: `controller.py`, `run_controller.py`, `topology.py`, `server.py`, `client.py`, `protocol.py`, `experiment.py`, `demo_t7.py`, `run_all.sh`, `analyze.py`, `make_report.py`, `tests/`, `results/` (raw per-run data, controller logs, tables, charts), `README.md`.

Dependencies must be installed inside the virtual environment. Two setup errors seen during the project are shown in Appendix B: run_all.sh must be made executable with chmod +x, and starting the controller outside the activated venv fails with ModuleNotFoundError: No module named os_ken.

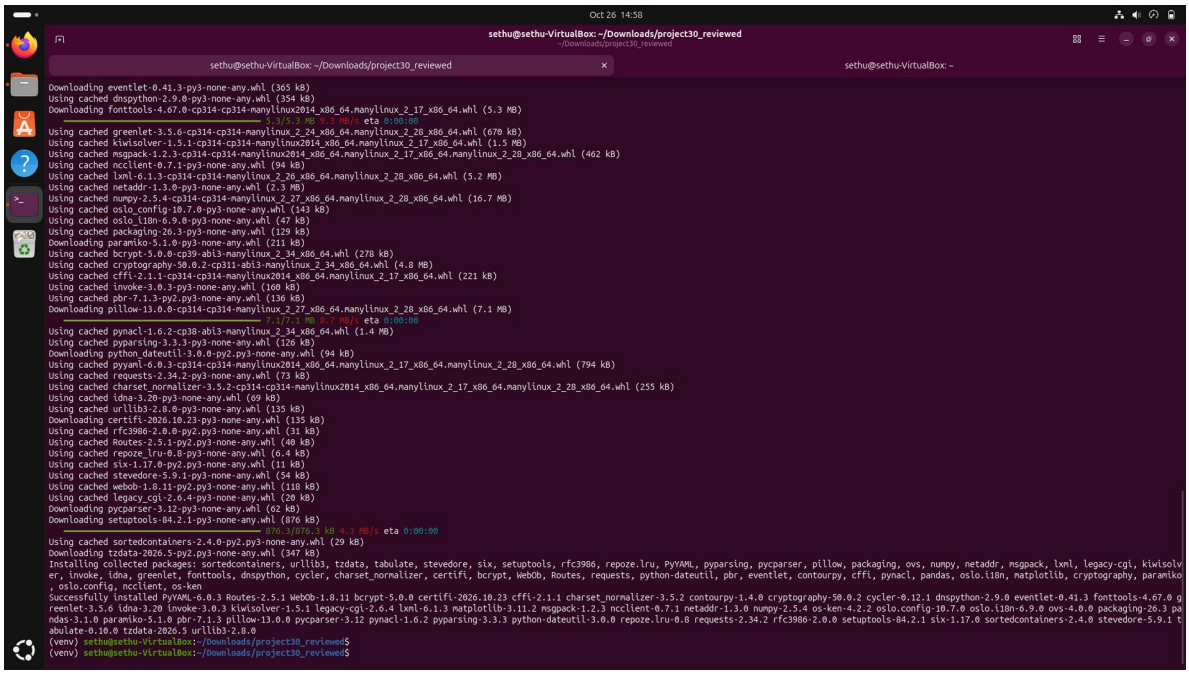


Figure 20. Dependency installation in the virtual environment: pip finishes with os-ken 4.2.2, matplotlib, pandas, numpy and tabulate installed.

11. Conclusion

The system keeps a UDP application running through a link failure and through a silent black-hole, where a static network loses about 30 % of the traffic and stays down for the whole failure. Combining OpenFlow link events with reports from the application gives the controller visibility into failures that the network alone cannot observe, and the make-before-break reroute keeps the recovery time down to the detection delay. The evaluation (90 runs, 5 repetitions per configuration, significance tests, and a documented list of problems found along the way) shows a clear and reproducible advantage over the static baseline, at no cost when the network is healthy.

Appendix A – Mapping to the evaluation rubric

Rubric component	Marks	Where it is covered
Problem understanding and architecture	2	Sections 1 and 2
TCP/UDP socket implementation	4	Sections 2.4, 3.1, 3.2 (UDP, ACK/retransmit, concurrency, error handling)
Mininet and SDN implementation	4	Sections 2.1, 3.3
Socket–SDN integration	3	Sections 2.3, 3.4
Initial testing and demo	2	Section 8
Complete application functionality	4	Sections 3.1, 3.2, 8
Dynamic SDN functionality	6	Section 3.3 (rerouting, recovery, monitoring)
Challenging scenario and robustness	4	Scenarios `linkdown` and `blackhole`, sections 5, 6, 7
Performance evaluation and comparison	5	Sections 4, 5, 6 (baseline, several loads, repetitions, significance)
Documentation, demo and viva	6	This report, README, section 10, section 7

Appendix B – Additional session screenshots

These screenshots document the working session. Those marked superseded show intermediate output that was later corrected; none of their values is used in the results above.

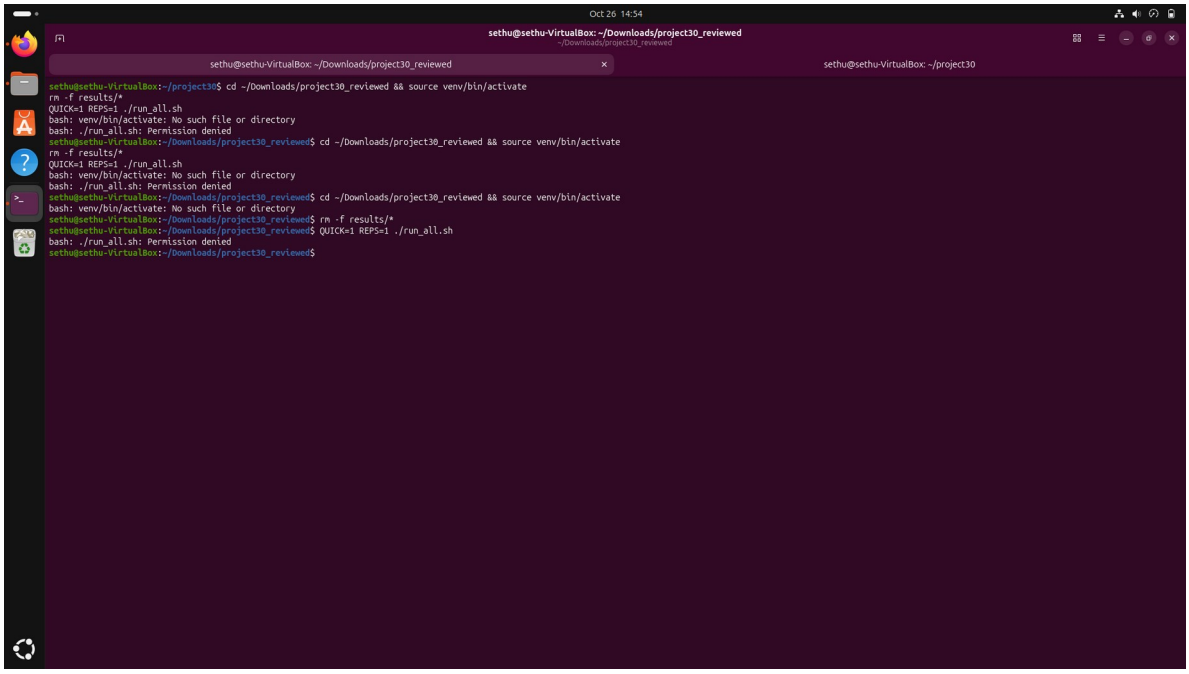


Figure 21. Setup errors at the start of the session: ./run_all.sh fails with "Permission denied" (fixed with chmod +x run_all.sh) and venv/bin/activate does not exist yet (the virtual environment was created afterwards).

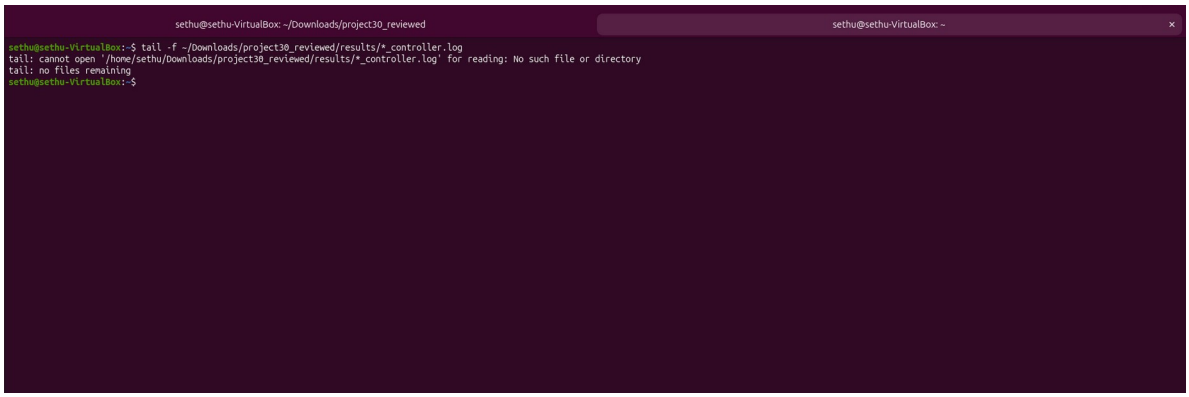


*Figure 22. tail -f on results/*_controller.log fails because no run has started yet and no log files exist.*

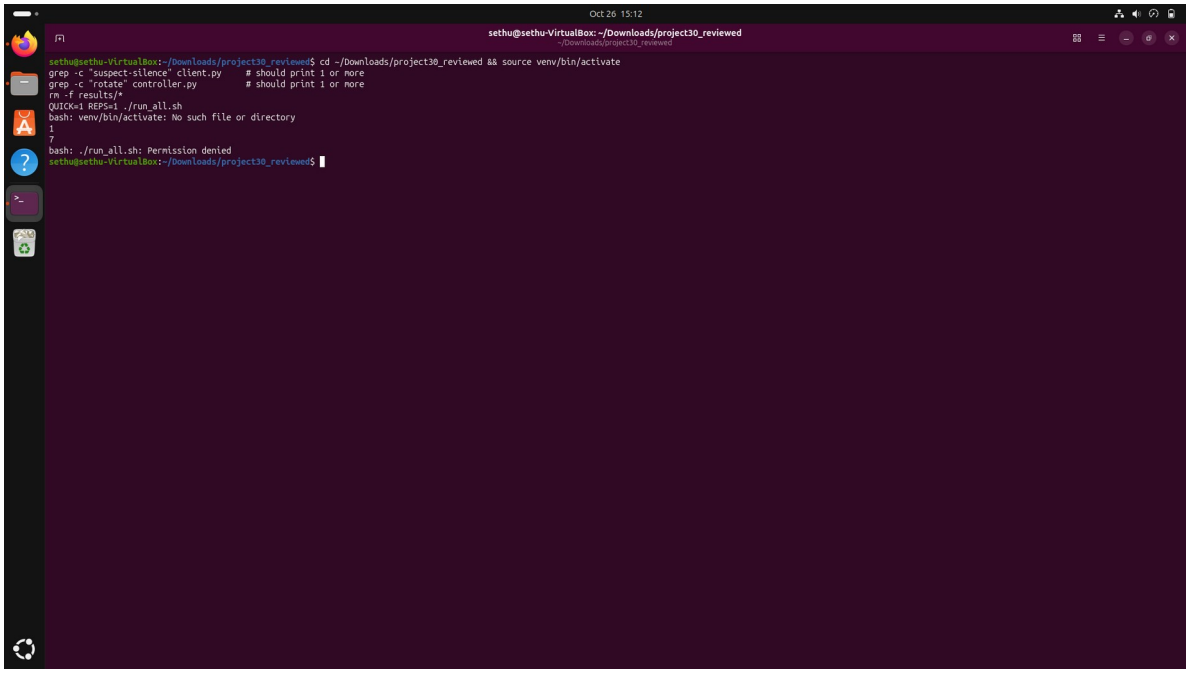


Figure 23. Pre-run check that the fixes are present in the code (suspect-silence found once in client.py, rotate found seven times in controller.py); the run itself again fails because the venv was not active and the script was not executable.

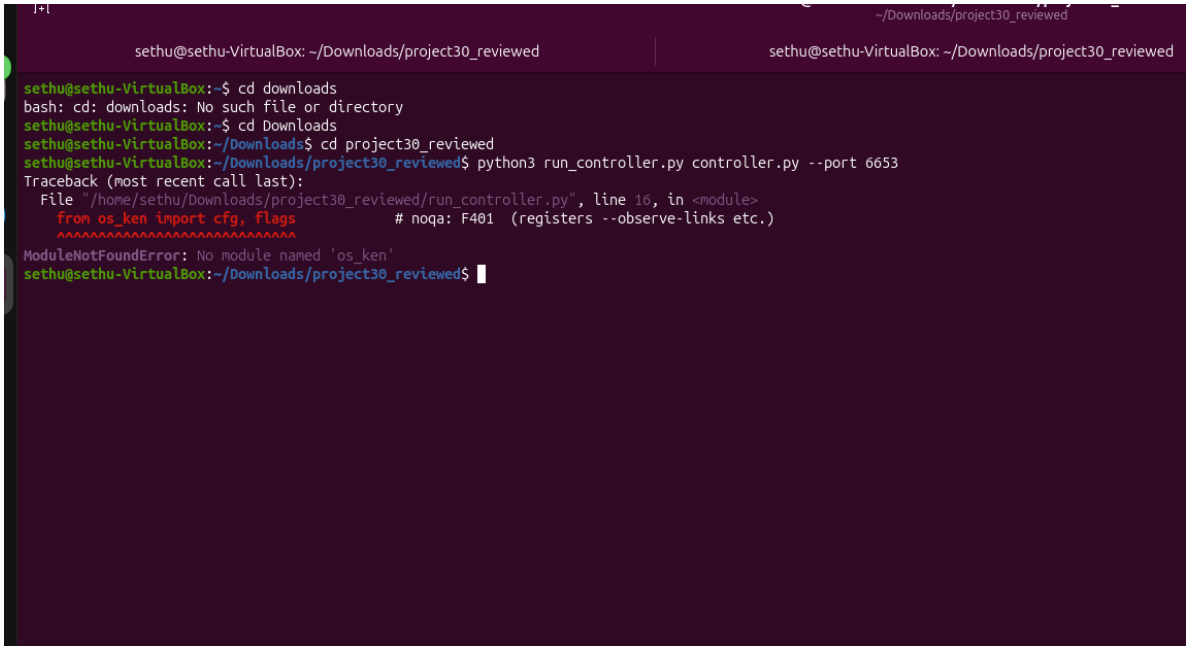


Figure 24. Starting the controller outside the virtual environment fails with ModuleNotFoundError: No module named os_ken; activating the venv fixes it.

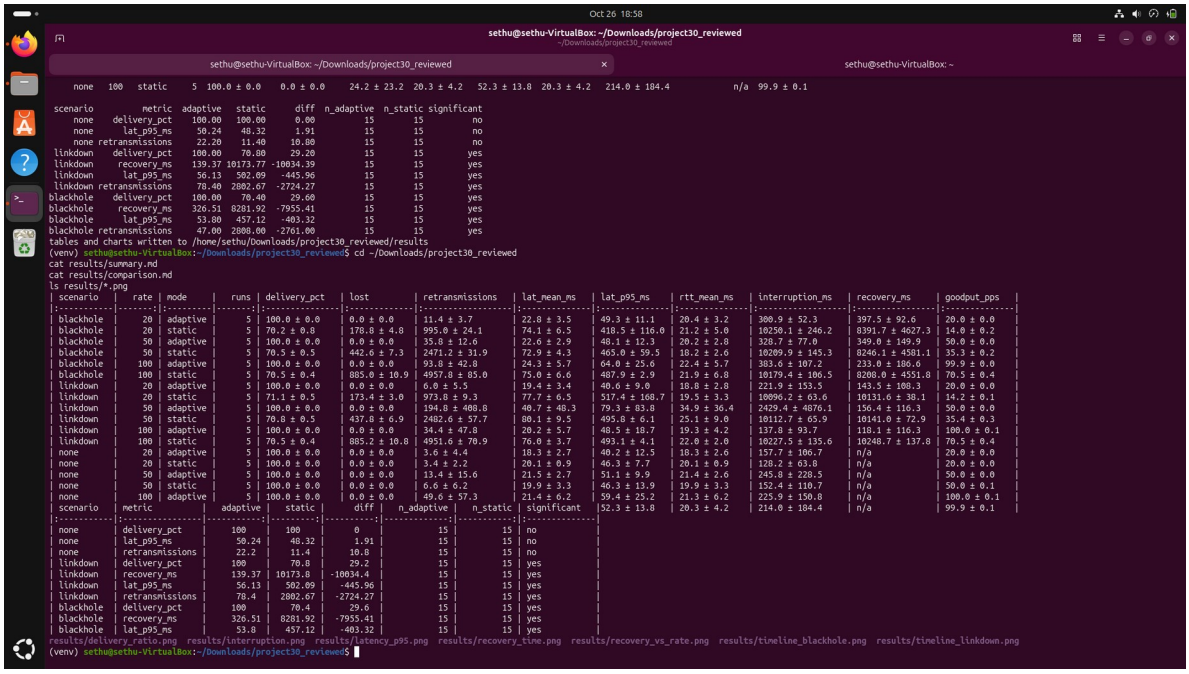


Figure 25. Superseded: analysis before the 11 s outlier run was repeated. The link-down, 50 packets/s adaptive row still contains it (retransmissions 194.8 ± 408.8, interruption 2 429 ± 4 876 ms) and the pooled link-down recovery is 139.37 ms.

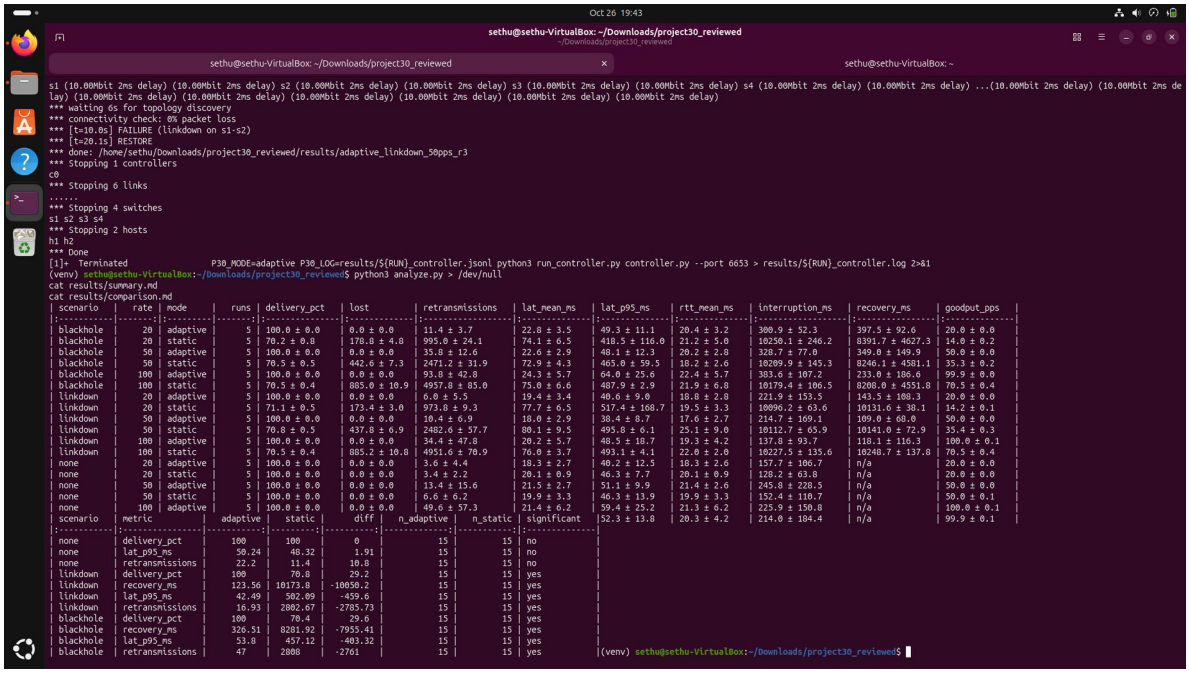


Figure 26. Superseded: analysis after the outlier run was repeated (link-down recovery 123.56 ms) but before the sustained-recovery rule, so black-hole recovery is still 326.51 ms (adaptive) and 8 281.92 ms (static).

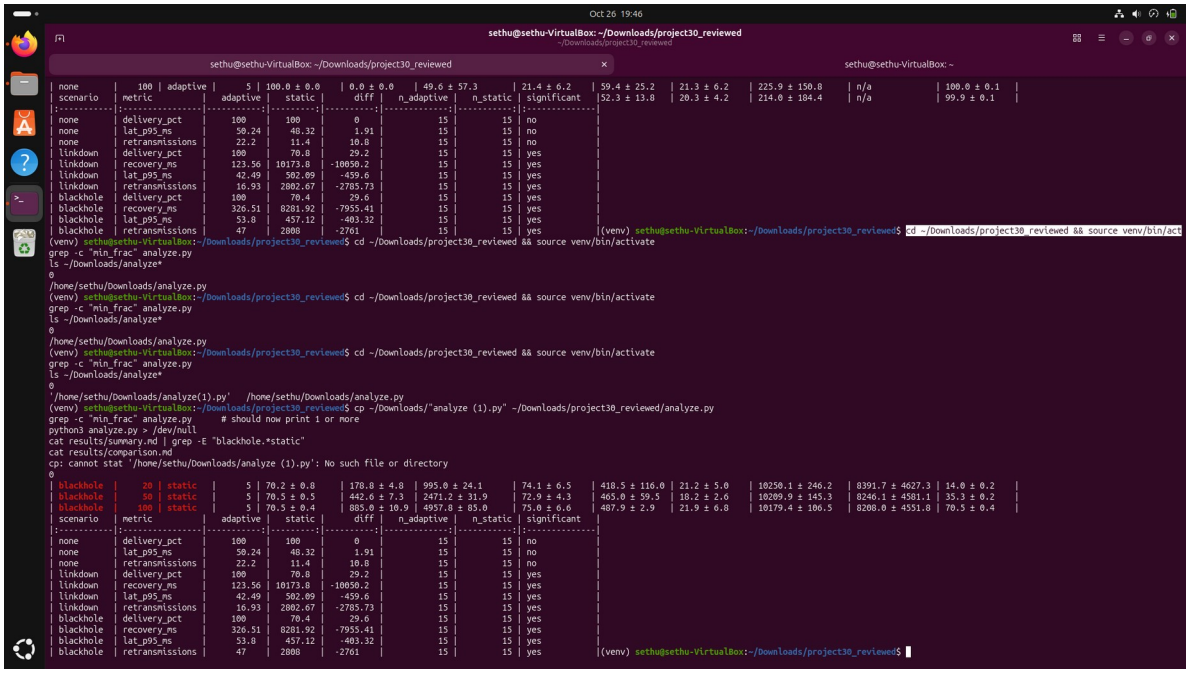


Figure 27. Superseded: re-running the analysis while analyze.py was being patched (min_frac check); the black-hole recovery values are still the old ones.